MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Virtual Networking

Unit 8 · Virtualization & Software Defined Data Center (10-150-130)

MID-STATE TECHNICAL COLLEGE

Learning Outcomes

By the end of this unit, you will be able to:



Build host networking

Configure Linux bridges and bonds and apply changes without losing the node.



Segment with VLANs

Use VLAN-aware bridges to separate management, storage, cluster, and guest traffic.



Secure virtual networks

Build firewall policy with security groups, aliases, and IP sets at every level.



Deploy SDN overlays

Create zones, VNets, and subnets, and explain VXLAN, EVPN, and fabrics.



Design for performance

Choose vNIC models, MTU, and offload options for a multi-tier design.

LESSON 1

Bridges, Bonds & Host Networking



The Bridged Model

- A **Linux bridge** (`vmbrN`) is a virtual switch: guest vNICs and a physical NIC plug into it as ports.
- The installer creates `vmbr0` on the first NIC — every VM on it appears on the LAN with its own MAC.
- A bridge with `bridge-ports none` is an **isolated switch** — guests reach each other but nothing else.
- Each host supports up to 4094 bridges; one per traffic class is the usual pattern.



Naming conventions

`eno1`, `enp3s0f1` — physical NICs

`vmbr0` — bridge

`bond0` — bond

`eno1.50`, `bond0.30` — VLAN 50 / 30

The name tells you the device type — a big help when troubleshooting.

Reading /etc/network/interfaces

The default configuration on a fresh Proxmox VE install — one NIC, one bridge, one address:

```
iface eno1 inet manual

auto vmbr0
iface vmbr0 inet static
        address 10.22.3.11/24
        gateway 10.22.2.1
        bridge-ports eno1
        bridge-stp off
        bridge-fd 0
```

- The physical NIC is `manual` — it carries no IP; the **host address lives on the bridge**.
- `bridge-ports` names the uplink; `bridge-stp off` and `bridge-fd 0` skip spanning-tree delays.

Applying Changes Safely

GUI (preferred)

- Edits are staged in `/etc/network/interfaces.new`
- Review pending changes, then click **Apply Configuration**
- **ifupdown2** reloads live — no reboot required

CLI

- Edit `/etc/network/interfaces` directly
- Apply with `ifreload -a`
- Avoid `ifdown vmbr0` — guests are not reconnected by `ifup`

Upgrades can rename NICs: pve-network-interface-pinning generate pins names like nic1, nic2 so a new systemd naming scheme cannot orphan your bridge.

Lost the node anyway? Use the iLO5 remote console to fix the file — that is exactly what out-of-band management is for.

Bridged, Routed, or NAT?

Choose the model that matches what the upstream network will tolerate:

Model	How it works	Use when
Bridged	Guests share L2 with the LAN; each has its own MAC	On-premises LAN with an external gateway — the default
Routed	Host routes for guests; one MAC on the wire	Provider blocks extra MACs but gives you an IP block
Masquerade (NAT)	Guests on a private subnet; host rewrites source IP	Only one public IP; inbound needs port forwarding

The pods use the bridged model; every guest you build in Lab 12 is directly reachable on its VLAN.

Linux Bonds

A bond binds several NICs into one logical interface. Seven modes; two you will actually use:

Mode	Behavior	Notes
`802.3ad`	LACP — all links active, hashed per flow	Recommended when the switch supports LACP
`active-backup`	One link active; the other takes over on failure	Recommended without switch support
`balance-rr`	Packets round-robin across links	Can reorder packets; rarely used
`balance-xor`	Link chosen by MAC hash	Needs static switch config
`balance-tlb` / `-alb`	Adaptive load balancing, no switch support	Do not use under a bridge with guests

One flow never exceeds one link's speed — LACP adds aggregate bandwidth, not single-stream speed.

A Bond as the Bridge Uplink

```
auto bond0
iface bond0 inet manual
        bond-slaves eno1 eno2
        bond-miimon 100
        bond-mode 802.3ad
        bond-xmit-hash-policy layer2+3

auto vmbr0
iface vmbr0 inet static
        address 10.22.3.11/24
        bridge-ports bond0
```

- `bond-miimon 100` checks link state every 100 ms.
- `layer2+3` hashes on MAC and IP to spread flows.



Corosync does not need a bond

Cluster traffic fails over between its own redundant links. Some bond modes cause Corosync instability — give it two separate networks instead.

LESSON 2

VLANs & Traffic Segmentation



VLAN Fundamentals

- A **VLAN** (IEEE 802.1Q) is an isolated broadcast domain identified by a 12-bit **tag** — IDs 1–4094.
- **Tagged** frames carry the ID; an **untagged** (native/access) port assumes one VLAN.
- A hypervisor uplink is usually a **trunk** carrying many VLANs to the physical switch.
- VLANs cut broadcast domains and separate traffic — but routing between them happens elsewhere.



Why separate traffic?

Management — protect the GUI and SSH

Storage — jumbo frames, no contention

Cluster — Corosync hates latency

Guest — tenants and tiers isolated

Four Ways to Do VLANs in Proxmox VE

The tag can be applied at the bridge, by Open vSwitch, or inside the guest:

Method	How it works	Trade-off
VLAN-aware bridge	One bridge; the tag is set per vNIC and handled transparently	Simplest and recommended
Traditional	`eno1.5` + `vmbr0v5` created per VLAN on demand	Extra interfaces linger until reboot
Open vSwitch	OVS internal ports and tagged ports	Extra package; more features
Guest-configured	Guest tags its own frames on a trunk vNIC	Many VLANs on one vNIC; host cannot enforce

Rule of thumb: put the VLAN on the interface with the fewest layers between it and the physical NIC.

Configuring a VLAN-Aware Bridge

```
auto vmbr0
iface vmbr0 inet manual
        bridge-ports bond0
        bridge-stp off
        bridge-fd 0
        bridge-vlan-aware yes
        bridge-vids 2-4094

auto vmbr0.5
iface vmbr0.5 inet static
        address 10.10.10.2/24
```

What each piece does

`bridge-vlan-aware yes` — the bridge filters by tag

`bridge-vids` — VLANs allowed on the trunk

`vmbr0.5` — host IP on VLAN 5

`tag=20` — this vNIC is an access port on VLAN 20

- Guest side: `qm set 101 --net0 virtio,bridge=vmbr0,tag=20`

Designing the Traffic Classes

Networks to separate

- **Management** — GUI on 8006, SSH, API
- **Storage** — iSCSI/NFS to TrueNAS at MTU 9000
- **Cluster** — Corosync, low latency, redundant links
- **Guest** — one VLAN per tier or tenant

Design rules

- Match MTU on every hop — host, switch, and SAN
- Keep storage and Corosync off guest-facing links
- Document VLAN IDs and subnets before you build
- Reference storage and cluster networks by IP, not DNS

```
ping -M do -s 8972 <SAN storage IP>     # 8972 + 28 header bytes = 9000
```

If the large ping fails but a normal ping works, something in the path is still at MTU 1500.

VLANs Are Not a Firewall

- **VLAN hopping** — double-tagged frames or switch spoofing can escape a VLAN on a poorly configured trunk.
- Never leave guests on the **native VLAN**; tag everything on the trunk.
- Limit `bridge-vids` to the VLANs a host actually needs.
- Disable dynamic trunk negotiation on the physical switch ports.



SECURITY FOCUS

A VLAN separates broadcast domains; it does not decide who may talk to whom once traffic is routed.

Why it matters: Treating segmentation as security leaves every routed path between VLANs wide open.



LESSON 3

The Proxmox VE Firewall

Firewall Architecture

Rules live at four levels, all stored on the cluster file system and replicated to every node:

Level	Configuration file	Protects
Datacenter	`/etc/pve/firewall/cluster.fw`	Global options, groups, aliases, IP sets
Node	`/etc/pve/nodes/<node>/host.fw`	The Proxmox host itself
VM / CT	`/etc/pve/firewall/<VMID>.fw`	Each guest NIC with firewall=1
VNet (SDN)	`/etc/pve/sdn/firewall/<vnet>.fw`	Forwarded traffic — nftables only

The firewall is stateful and supports IPv4 and IPv6; rules are enforced only on NICs with the firewall enabled.

Turning It On Without Locking Yourself Out

- The firewall is **disabled by default** cluster-wide — enable it at Datacenter → Firewall → Options.
- Before enabling, allow `8006/tcp` (GUI) and `22/tcp` (SSH) from the management subnet.
- Cluster ports: Corosync `5405-5412/udp`; live migration `60000-60050/tcp`.
- Guests need `firewall=1` on each vNIC plus the VM-level firewall enabled.

Keep a way back in: Open an SSH session before enabling. If you lock yourself out, run pve-firewall stop from the iLO5 console.

Reusable Rule Objects

Write rules once and reference them by name — never copy the same IP into fifty rules:

Object	What it is	Referenced as
Security group	A named bundle of rules applied to many guests	`GROUP webserver`
Alias	A readable name for one IP or network	`-source mgmt-net`
IP set	A list of addresses or networks (can exclude)	`-source +admins`
Macro	A built-in service definition	`SSH`, `HTTPS`, `DNS`

Change the alias or IP set once, and every rule that references it updates across the cluster.

A Guest Firewall File

```
# /etc/pve/firewall/101.fw
[OPTIONS]
enable: 1
policy_in: DROP
macfilter: 1

[RULES]
GROUP webserver
IN SSH(ACCEPT) -source +admins -log info
```

Reading it

`policy_in: DROP` — default deny inbound

`macfilter: 1` — guest cannot spoof another MAC

`GROUP webserver` — HTTP/HTTPS from the group

`+admins` — SSH only from the admin IP set

nftables: proxmox-firewall

- `proxmox-firewall` is a Rust re-implementation of the firewall on **nftables** instead of iptables.
- Enable per host: Host → Firewall → Options → **nftables** (or `nftables: 1` in `host.fw`).
- Required for **forward rules** and the **VNet firewall** — zone-based policy for SDN.
- Still flagged as tech preview; inspect with `nft list ruleset` or `nft monitor trace`.



SECURITY FOCUS

Turning on the nftables option without the proxmox-firewall package installed generates no rules at all.

Why it matters: The GUI looks protected while every host and guest is wide open.

Micro-Segmentation & Zero Trust

- **Micro-segmentation** wraps each workload in its own policy instead of trusting a whole subnet.
- **Zero trust** assumes the network is hostile — every connection is allowed explicitly or not at all.
- In Proxmox VE: per-VM `policy_in: DROP` plus security groups is micro-segmentation.
- **East-west** traffic between VMs never touches the perimeter firewall — only host-level rules see it.

Defense in depth

Layer the controls: physical switch ACLs, VLANs, datacenter rules, VM rules, then the guest OS firewall. Any single layer will eventually be misconfigured.

LESSON 4

Software-Defined Networking



What Makes Networking "Software-Defined"

Traditional networking

- Each device decides forwarding on its own
- Configured box by box, by hand
- Topology is the physical cabling

Software-defined

- **Control plane** centralized; **data plane** stays distributed
- Declared once, pushed by API or controller
- **Overlay** networks ride on an **underlay**

Overlay vs. underlay: the underlay is the physical routed network; overlays such as VXLAN tunnel virtual L2 segments across it.

OpenFlow is the textbook SDN protocol; Proxmox VE SDN instead builds on standard Linux networking and FRRouting (BGP, OSPF, OpenFabric).

Proxmox VE SDN Building Blocks

- A **zone** is a separated network area and defines the technology (VLAN, VXLAN, EVPN…).
- A **VNet** lives in a zone and appears as a normal bridge on every node — assign it to guests.
- A **subnet** is an IP range in a VNet: gateway, SNAT, DHCP ranges, IPAM registration.
- Edits are **pending** until you click Apply in Datacenter → SDN — rolled out atomically.



Support status (PVE 9.x)

Supported: core SDN, zones, VNets

Tech preview: IPAM with DHCP (dnsmasq), FRR routing and controllers

Config lives in `/etc/pve/sdn` on pmxcfs

Zone Types

The zone type decides how a VNet is isolated and how far it can reach:

Zone	How it isolates	Typical use
Simple	Isolated bridge local to each node; NAT or routed	Lab networks, single-node NAT
VLAN	Tags VNet traffic on an existing bridge	VMs talking across nodes on a VLAN fabric
QinQ	Stacked tags — service VLAN outside, VNet inside	Providers isolating tenants (switches must support it)
VXLAN	L2 frames in UDP tunnels over the underlay	L2 across racks or sites without new VLANs
EVPN	VXLAN plus BGP for routing and anycast gateways	Routed, multi-tenant, multi-cluster networks

EVPN is the only zone that needs a controller; Simple is the only zone with automatic DHCP today.

Overlays and MTU

- **VXLAN** wraps Ethernet frames in UDP, destination port `4789`.
- Encapsulation adds **50 bytes** — guests on a 1500 underlay use MTU `1450` (the zone default).
- QinQ adds 4 bytes; run the underlay at MTU 9000 and the overhead stops mattering.
- A mismatched MTU looks like "ping works, file copies hang" — test with `ping -M do`.



SECURITY FOCUS

VXLAN provides no encryption. Joining sites requires IPsec, a site-to-site VPN, or a WireGuard fabric underneath.

Why it matters: An overlay across an untrusted link exposes every guest frame in clear text.

New in Proxmox VE 9

The SDN stack grew from virtual networks into building the underlay itself:

Feature	Release	Why it matters
SDN Fabrics (OpenFabric, OSPF)	9.0	Spine-leaf underlay built from the GUI instead of FRR files
OSPF redistribution, more EVPN controllers	9.1	Fabric routes and EVPN integrate with existing networks
WireGuard and BGP fabrics; route maps	9.x	Encrypted underlays and route policy from the datacenter
Interface name pinning tool	9.x	NIC names stay stable across upgrades

Proxmox VE 9 is based on Debian 13 “Trixie”; check the roadmap release notes for the exact version on your pod.

Building a VNet

VLAN zone — across nodes

- Zone `labvlan`, bridge `vmbr0`
- VNet `vnet20`, tag `20`
- Apply — vnet20 appears on every node
- VMs on different hosts share VLAN 20

Simple zone — isolated + NAT

- Zone `isolated`, no bridge needed
- Subnet `172.16.0.0/24`, gateway `.1`
- **SNAT** on — the node masquerades
- Optional DHCP via `dnsmasq`

Same result as a hand-edited bridge — but defined once for the whole cluster and permission-controlled per zone.

Linux Bridge, OVS, or SDN VNet?

	Linux bridge	Open vSwitch	SDN VNet
Install	Built in	`openvswitch-switch` package	Built in since 8.1
Scope	One node	One node	Whole cluster
Strength	Simple, fast, well understood	Flow rules, OpenFlow, OVS bonds	Central definition, overlays, IPAM
Use when	Most hosts — the default	You need a specific OVS feature	Many nodes, tenants, or sites

Recommendation: start with Linux bridges and SDN VNets; add OVS only for a feature you can name.

Network Function Virtualization

- **NFV** runs network services — routers, firewalls, load balancers — as software on commodity servers.
- On Proxmox VE: an OPNsense, pfSense, or VyOS VM with one vNIC per VLAN, or one tagged trunk.
- **Service chaining** sends traffic through several functions in order: firewall → IDS → load balancer.
- Gains: snapshots, HA, and live migration for the network itself. Cost: CPU and careful sizing.



Router VM pattern

`net0` — WAN on `vmbr0` (untagged)

`net1` — trunk on `vmbr1` (no tag)

The guest builds its own VLAN sub-interfaces and routes between tiers.

LESSON 5

Performance & Network Design



Choosing a vNIC Model

The emulated hardware the guest sees decides most of its network performance:

Model	Type	Use when
VirtIO	Paravirtualized	Always, if the guest has drivers — the default choice
Intel E1000 / E1000E	Emulated	Guest has no VirtIO driver yet (early Windows installs)
VMware vmxnet3	Emulated	Temporary, for imported guests until drivers are swapped
Realtek RTL8139	Emulated	Legacy operating systems only

```
qm set 101 --net0 virtio,bridge=vmbr0,tag=20,queues=4     # multiqueue: up to vCPU count
```

Beyond VirtIO: Offload & Acceleration

- **SR-IOV** splits one physical NIC into many virtual functions handed straight to guests.
- **PCI passthrough** gives a guest a whole NIC — near line rate, but the VM is pinned to that host.
- **DPDK** moves packet processing into user space for very high packet rates (NFV appliances).
- **Jumbo frames** (MTU 9000) cut per-packet overhead on storage and migration networks.



The trade-off

SR-IOV and passthrough need IOMMU (Intel VT-d or AMD-Vi) and break live migration. Use them only where the speed is worth losing mobility.

Multi-Tier Network Design

Each tier gets its own segment and a policy that allows only the flows the design requires:

Tier	VLAN	Allowed inbound	Notes
DMZ / Web	20	HTTPS from anywhere	Only tier exposed to users
Application	30	App port from Web tier only	No internet inbound
Database	40	DB port from App tier only	Most restrictive policy
Management	10	GUI and SSH from admin IP set	Never routed to guests
Storage / Cluster	50 / 60	Hosts and SAN only	MTU 9000; not guest-facing

VLAN IDs are illustrative — document your own scheme before building.

Securing the Virtual Network

SECURITY FOCUS

Treat every guest as potentially compromised: default-deny each VM, keep management and storage off guest networks, enable MAC and IP filtering, and log what you drop.

Why it matters: One breached VM on a flat network can reach the hypervisor GUI, the SAN, and every neighbor.

- **Segment** — VLANs or VNets per tier, management isolated, trunks tagged.
- **Filter** — datacenter, host, VM, and VNet rules; `macfilter` and `ipfilter` on.
- **Encrypt** — VPN or WireGuard under any overlay crossing an untrusted link.

Hands-On Lab 12: Networks Configuration

NETLAB Lab 12 · your WR/SP pod, Host1–Host3 · details and due date in Blackboard Ultra



- Create and verify Linux bridges and a bond on each host.
- Make vmbr0 VLAN-aware and place guests on tagged VLANs.
- Build a VLAN zone and VNet in SDN and test across nodes.
- Enable the firewall with management access preserved; apply a security group.
- Confirm the SAN path still passes jumbo frames end to end.

Graded on the live pod — working connectivity and policy, not screenshots.

Key Takeaways

- A bridge is a switch; the host IP lives on the bridge, not the NIC.
- VLAN-aware bridges segment traffic — but segmentation is not security.
- Default-deny every guest; reuse groups, aliases, and IP sets.
- SDN defines zones and VNets once for the cluster; mind the overlay MTU.
- VirtIO by default; jumbo frames for storage; offloads cost mobility.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 8 · Virtual Networking

MID-STATE TECHNICAL COLLEGE